

宁波大学
NINGBO UNIVERSITY

实 验 报 告

[实验 13 虚拟化]

姓名：	顾凯琪
学号：	226001624
课程名称：	操作系统
所在学院：	信息科学与工程学院
专业班级：	22 计算机科学与技术 2 班
任课教师：	江先亮

实验项目名称	实验 13 虚拟化

一、 实验目的与要求：

（1）了解 openEuler 常见的虚拟化方法

（2）基于 openEuler 部署 QEMU 和 Docker

二、实验环境：

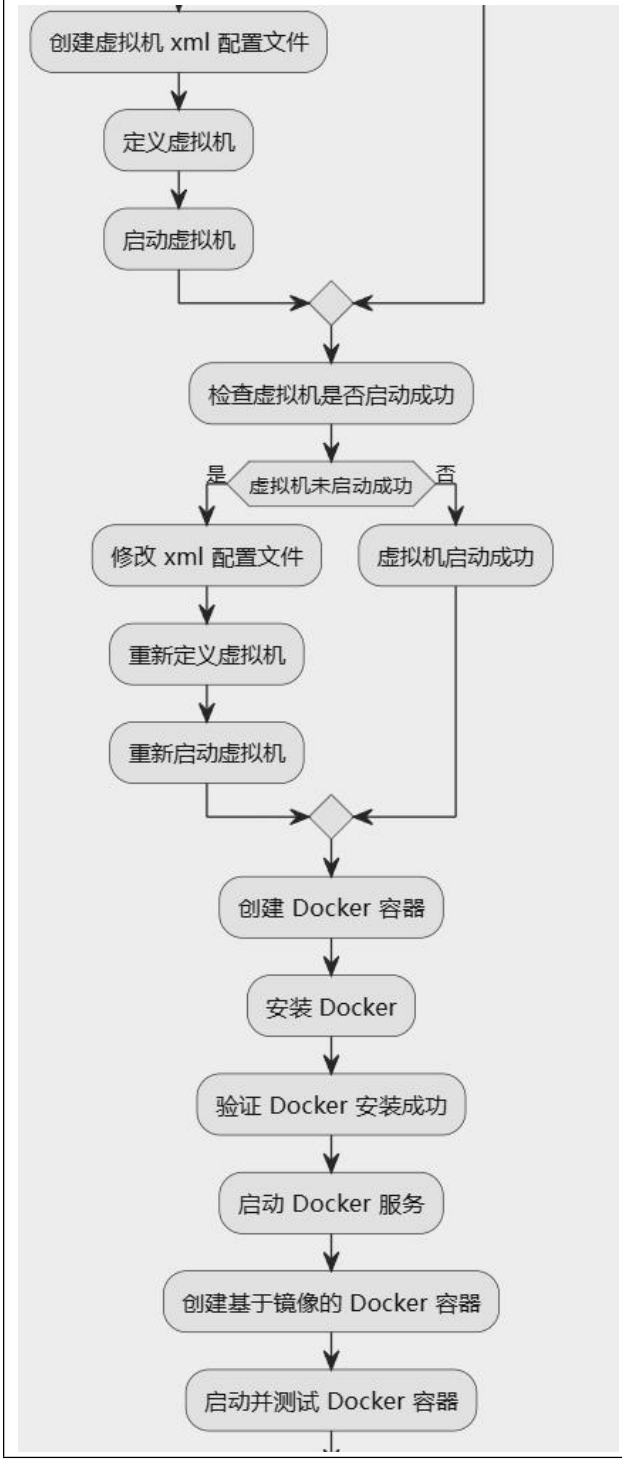
Win11 OS

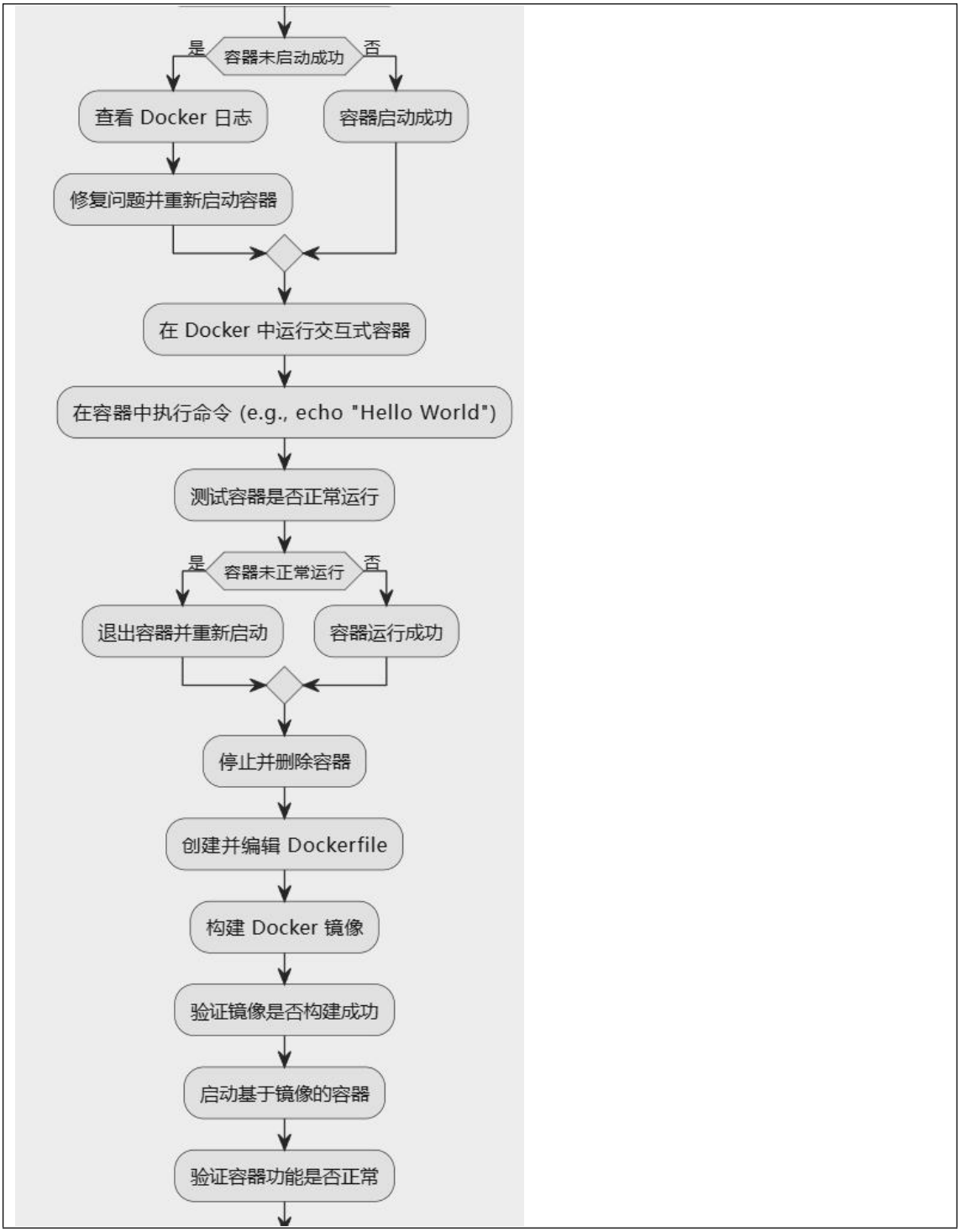
VMware workstation 17 pro

OpenEular 22.03

FinalShell 4.5.6

二、　实验方法（原理、流程图）


创建虚拟机 xml 配置文件
定义虚拟机
启动虚拟机
检查虚拟机是否启动成功
是
虚拟机未启动成功
否
修改 xml 配置文件
虚拟机启动成功
重新定义虚拟机
重新启动虚拟机
创建 Docker 容器
安装 Docker
验证 Docker 安装成功
启动 Docker 服务
创建基于镜像的 Docker 容器
启动并测试 Docker 容器

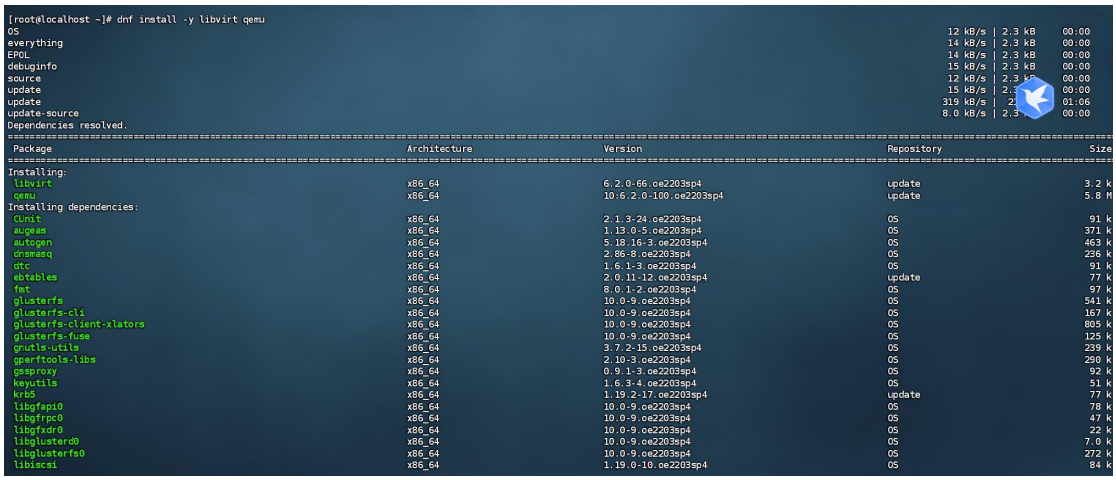

是
容器未启动成功
否
查看 Docker 日志
容器启动成功
修复问题并重新启动容器
在 Docker 中运行交互式容器
在容器中执行命令 (e.g., echo "Hello World")
测试容器是否正常运行
是
容器未正常运行
否
退出容器并重新启动
容器运行成功
停止并删除容器
创建并编辑 Dockerfile
构建 Docker 镜像
验证镜像是否构建成功
启动基于镜像的容器
验证容器功能是否正常

四、实验过程、步骤及内容

任务 1

使用 **libvirt+xml** 配置文件的方法，搭建 **QEMU** 虚拟机运行环境。

(1)安装虚拟化组件：



```
[root@localhost ~]# dnf install -y libvirt qemu
OS                                                                                                  12 kB/s | 2.3 kB     00:00
everything                                                                                          14 kB/s | 2.3 kB     00:00
EPOL                                                                                                14 kB/s | 2.3 kB     00:00
debuginfo                                                                                           15 kB/s | 2.3 kB     00:00
source                                                                                              12 kB/s | 2.3 kB     00:00
update                                                                                              15 kB/s | 2.3 kB     00:00
update                                                                                             319 kB/s |  21 kB     01:06
update-source                                                                                      8.0 kB/s | 2.3 kB     00:00
Dependencies resolved.
================================================================================================================================
 Package                      Architecture          Version                          Repository                           Size
================================================================================================================================
Installing:
 libvirt                      x86_64                6.2.0-66.oe2203sp4               update                              3.2 k
 qemu                         x86_64                10:6.2.0-100.oe2203sp4           update                              5.8 M
Installing dependencies:
 CUnit                        x86_64                2.1.3-24.oe2203sp4               OS                                   91 k
 augeas                       x86_64                1.13.0-5.oe2203sp4               OS                                  371 k
 autogen                      x86_64                5.18.16-3.oe2203sp4              OS                                  463 k
 dnsmasq                      x86_64                2.86-8.oe2203sp4                 OS                                  236 k
 dtc                          x86_64                1.6.1-3.oe2203sp4                OS                                   91 k
 ebtables                     x86_64                2.0.11-12.oe2203sp4              update                               77 k
 fmt                          x86_64                8.0.1-2.oe2203sp4                OS                                   97 k
 glusterfs                    x86_64                10.0-9.oe2203sp4                 OS                                  541 k
 glusterfs-cli                x86_64                10.0-9.oe2203sp4                 OS                                  167 k
 glusterfs-client-xlators     x86_64                10.0-9.oe2203sp4                 OS                                  805 k
 glusterfs-fuse               x86_64                10.0-9.oe2203sp4                 OS                                  125 k
 gnutls-utils                 x86_64                3.7.2-15.oe2203sp4               OS                                  239 k
 gperftools-libs              x86_64                2.10-3.oe2203sp4                 OS                                  290 k
 gssproxy                     x86_64                0.9.1-3.oe2203sp4                OS                                   92 k
 keyutils                     x86_64                1.6.3-4.oe2203sp4                OS                                   51 k
 krb5                         x86_64                1.19.2-17.oe2203sp4              update                               77 k
 libgfapi0                    x86_64                10.0-9.oe2203sp4                 OS                                   78 k
 libgfrpc0                    x86_64                10.0-9.oe2203sp4                 OS                                   47 k
 libgfxdr0                    x86_64                10.0-9.oe2203sp4                 OS                                   22 k
 libglusterd0                 x86_64                10.0-9.oe2203sp4                 OS                                  7.0 k
 libglusterfs0                x86_64                10.0-9.oe2203sp4                 OS                                  272 k
 libiscsi                     x86_64                1.19.0-10.oe2203sp4              OS                                   84 k
```

(2) 验证安装是否成功:

①首先查看内核是否支持 **KVM** 虚拟化，即验证**/dev/kvm** 和 **/sys/module/kvm** 文件是否存在。注意，在这一步中需要将内核切换成 **openeuler** 原本的内核：



```
[root@localhost ~]#
[root@localhost ~]# ls /dev/kvm
ls: 无法访问 '/dev/kvm': No such file or directory
[root@localhost ~]# ls /sys/module/kvm
ls: 无法访问 '/sys/module/kvm': No such file or directory
[root@localhost ~]#
```

②确认 **QEMU** 是否安装成功。若安装成功则可以看到 **QEMU** 软件包信息：

```
[root@localhost ~]# rpm -qi qemu
Name        : qemu
Epoch       : 10
Version     : 6.2.0
Release     : 100.oe2203sp4
Architecture: x86_64
Install Date: 2024年12月11日 星期三 10时42分48秒
Group       : Unspecified
Size        : 21916318
License     : GPLv2 and BSD and MIT and CC-BY-SA-4.0
Signature   : RSA/SHA256, 2024年10月15日 星期二 19时37分40秒, Key ID 3b477c60b675600b
Source RPM  : qemu-6.2.0-100.oe2203sp4.src.rpm
Build Date  : 2024年10月15日 星期二 19时31分05秒
Build Host  : dc-64g.compass-ci
Packager    : http://openeuler.org
URL         : http://www.qemu.org
Summary     : QEMU is a generic and open source machine emulator and virtualizer
Description :
QEMU is a FAST! processor emulator using dynamic translation to achieve good emulation speed.

QEMU has two operating modes:

   Full system emulati[redacted]is mode, QEMU emulates a full system (for example a PC),
   including one or several processors and various peripherals. It can be used to launch
   different Operating Systems without rebooting the PC or to debug system code.

   User mode emulation. In this mode, QEMU can launch processes compiled for one CPU on another CPU.
   It can be used to launch the Wine Windows API emulator (https://www.winehq.org) or to ease
   cross-compilation and cross-debugging.
You can refer to https://www.qemu.org for more infortmation.
[root@localhost ~]#
```

③确认 **libvirt** 是否安装成功：

```
[root@localhost ~]# rpm -qi libvirt
Name        : libvirt
Version     : 6.2.0
Release     : 66.oe2203sp4
Architecture: x86_64
Install Date: 2024年12月11日 星期三 10时42分48秒
Group       : Unspecified
Size        : 0
License     : LGPLv2+
Signature   : RSA/SHA256, 2024年07月17日 星期三 20时45分32秒, Key ID 3b477c60b675600b
Source RPM  : libvirt-6.2.0-66.oe2203sp4.src.rpm
Build Date  : 2024年07月17日 星期三 20时43分45秒
Build Host  : dc-64g.c[redacted]
Packager    : http://openeuler.org
URL         : https://libvirt.org/
Summary     : Library providing a simple virtualization API
Description :
Libvirt is a C toolkit to interact with the virtualization capabilities
of recent versions of Linux (and other OSes). The main package includes
the libvirtd server exporting the virtualization support.
[root@localhost ~]#
```

(3)启动 **libvirtd** 服务：

```
[root@localhost ~]# systemctl start libvirtd
[root@localhost ~]#
```

(4)编写虚拟机 **xml** 配置文件：

```
<domain type='kvm'>
  <name>openEulerVM</name>
  <memory unit='GiB'>8</memory>
   <vcpu>4</vcpu>
    <os>
          <type arch='aarch64' machine='virt'>hvm</type>
          <loader readonly='yes' type='pflash'>/usr/share/edk2/aarch64/QEMU_EFI-fff>pflash.raw</loader>
          <nvram>/var/lib/libvirt/qemu/nvram/openEulerVM.fd</nvram>
           </os>
            <features>
                  <acpi/>
                  <gic version='3'/>
                   </features>
                    <cpu mode='host-passthrough'>
                          <topology sockets='2' cores='2' threads='1'/>
                           </cpu>
                            <iothreads>1</iothreads>
                             <clock offset='utc'/>
                              <on_poweroff>destroy</on_poweroff>
                               <on_reboot>restart</on_reboot>
                                <on_crash>restart</on_crash>
                                 <devices>
                                        <emulator>/usr/libexec/qemu-kvm</emulator>
                                        <disk type='file' device='disk'>
                                                <driver name='qemu' type='qcow2' iothread="1"/>
                                                 <source file='/mnt/openEuler-image.qcow2'/>
                                                  <target dev='vda' bus='virtio'/>
                                                   <boot order='1'/>
                                            </disk>
                                            <disk type='file' device='cdrom'>
                                                    <driver name='qemu' type='raw'/>
                                                     <source file='/mnt/openEuler-20.03-LTS-aarch64-dvd.iso'/>
                                                      <readonly/>
                                                       <target dev='sdb' bus='scsi'/>
                                                        <boot order='2'/>
                                                </disk>
                                                <interface type='bridge'>
                                                        <source bridge='br0'/>
                                                         <model type='virtio'/>
                                                  </interface>
--
```

(5)可以使用以下 **virsh** 命令操作虚拟机：

[root@localhost ~]# virsh define vm_openEuler.xml #创建虚拟机

```
[root@localhost ~]# virsh define vm_openEuler.xml
错误：从 vm_openEuler.xml 定义域失败
错误：不支持的配置：Emulator '/usr/libexec/qemu-kvm' does not support arch 'aarch64'

[root@localhost ~]#
```

提示本机不支持相关虚拟化

[root@localhost ~]# virsh edit openEulerVM #定义/创建虚拟机后，再次修改 **xml** 文件的配置内容

```
[root@localhost ~]# virsh edit openEulerVM
错误：获得域 'openEulerVM' 失败
```

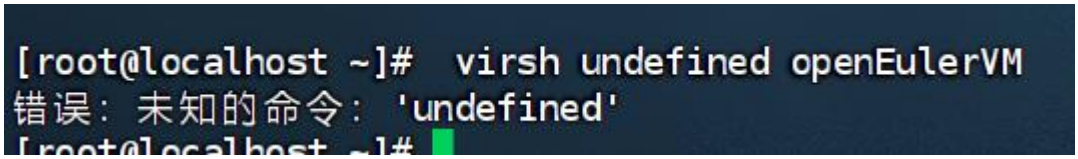
[root@localhost ~]# virsh start openEulerVM #启动虚拟机



[root@localhost ~]# virsh destroy openEulerVM #关闭虚拟机



[root@localhost ~]# virsh undefined openEulerVM #取消定义/删除虚拟机

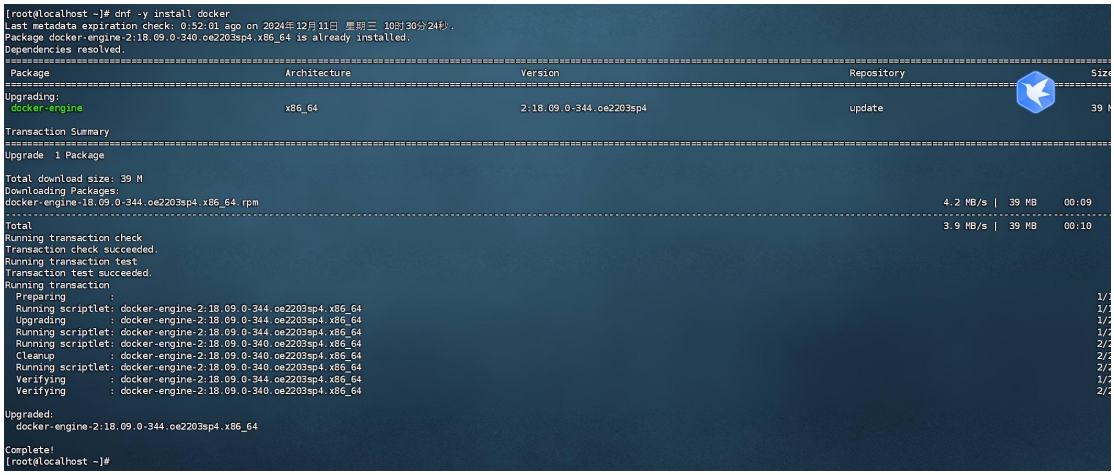


后续纠错尝试过程见五、实验数据（现象）处理分析

任务 2

安装并使用 **docker**，完成 **docker** 容器的新建、启动、守护态运行、终止与删除等操作。

(1)输入以下命令以安装 **docker**：



(2)启动 **docker:**

启动容器有两种方式:

①一种是基于镜像新建一个容器并启动;

②另外一个是将在终止状态（**stopped**）的容器重新启动。

例如，下面的命令输出一个 **“Hello World”**，之后终止容器:

首先 **pull** 下 **ubuntu** 镜像

```
[root@localhost ~]# docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
de44b265507a: Pull complete
Digest: sha256:80dd3c3b9c6cecb9f1667e9290b3bc61b78c2678c02cbdae5f0fea92cc6734ab
Status: Downloaded newer image for ubuntu:latest
[root@localhost ~]#
```

[root@localhost ~]# sudo docker run ubuntu /bin/echo 'Hello world'

```
[root@localhost ~]# sudo docker run ubuntu /bin/echo 'Hello world'
Hello world
[root@localhost ~]#
```

这跟在本地直接执行 **/bin/echo 'hello world'** 几乎感觉不出任何区别。

下面的命令则启动一个 **bash** 终端，允许用户进行交互；在交互模式下，用户可以通过所创建的终端来输入命令；输入 **exit** 可推出交

互模式。

[root@localhost ~]# sudo docker run -it ubuntu /bin/bash

```
[root@localhost ~]# sudo docker run -it ubuntu /bin/bash
root@146349a87e0e:/# exit
exit
[root@localhost ~]#
```

说明：**-t** 选项让 **Docker** 分配一个伪终端（**pseudo-tty**）并绑定到容器的标准输入上；**-i** 则让容器的标准输入保持打开。

(3)守护态运行

使用**-d** 参数可以让 **Docker** 容器在后台以守护态形式运行：

[root@localhost ~]# sudo docker run -d ubuntu /bin/sh -c “while true; do echo hello world; sleep 1; done”

```
[root@localhost ~]# sudo docker run -d ubuntu /bin/sh -c "while true; do echo hello world; sleep 1; done"
9987826f99b0c5a511e2a4de29aac74a08da32261e1a75dad2ec2f87d7fc5202
[root@localhost ~]#
```

(4) 使用 **docker stop container_id** 来终止一个运行中的容器。使用**"docker ps -n 2"**查看最后被创建的 **2** 个 **container_id**。

[root@localhost ~]# docker ps -n 2;

```
[root@localhost ~]# docker ps -n 2
CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS                      PORTS               NAMES
9987826f99b0        ubuntu              "/bin/sh -c 'while t…"   21 seconds ago      Up 21 seconds                                   epic_lichterman
146349a87e0e        ubuntu              "/bin/bash"              4 minutes ago       Exited (0) 3 minutes ago                        trusting_neumann
[root@localhost ~]#
```

(5)删除容器

使用 **docker rm container_id** 来删除一个处于终止状态的容器。

```
[root@localhost ~]# docker rm 146349a87e0e
146349a87e0e
[root@localhost ~]#
```

任务 3

编写 **Dockerfile** 文件，创建基于 **Ubuntu** 镜像，创建打印“**Hello world**”的 **docker** 镜像，并验证其可用性。

(1)新建空目录：

(2)转到新目录：

```
[root@localhost ~]# mkdir docker_test
[root@localhost ~]# ls
anaconda-ks.cfg  docker_test  exp1  exp3  exp5  exp8       file_demo2  file.txt
b'               exp          exp2  exp4  exp6  file_demo1  file_demo3  kernel-openEuler-22.03-l
[root@localhost ~]# cd docker_test/
```

(3)使用 **vi** 命令编辑 **Dockerfile** 文件：

```
[root@localhost docker_test]# vi Dockerfile
[root@localhost docker_test]#
```

(4)按下 **i** 键进入 **INSERT** 模式并输入以下代码：

```
#This dockerfile uses the ubuntu image
# VERSION 2 - EDITION 1
# Author: docker_user
# Command format: Instruction [arguments / command] ..
# Base image to use, this must be set as the first line
FROM ubuntu
# Maintainer: docker_user <docker_user at email.com> (@docker_user)
MAINTAINER docker_user docker_user@email.com
# Commands to update the image
RUN echo "deb http://archive.ubuntu.com/ubuntu/ raring main universe" >>
/etc/apt/sources.list
# Commands when creating a new container
CMD echo “Hello world!”
```

(5)创建镜像：

```
[root@localhost docker_test]# docker image build -t helloworld_ubuntu .
Sending build context to Docker daemon   2.56kB
Step 1/4 : FROM ubuntu
 ---> b1d9df8ab815
Step 2/4 : MAINTAINER docker_user docker_user@email.com
 ---> Running in 7b1689a19a82
Removing intermediate container 7b1689a19a82
 ---> 110d83f86d0d
Step 3/4 : RUN echo "deb http://archive.ubuntu.com/ubuntu/ focal main universe" >> /etc/apt/sources.list
 ---> Running in de162489bf30
Removing intermediate container de162489bf30
 ---> d41794ad0702
Step 4/4 : CMD echo "Hello world!"
 ---> Running in 4f0b16f2cf2a
Removing intermediate container 4f0b16f2cf2a
 ---> c23f1406a6f2
Successfully built c23f1406a6f2
Successfully tagged helloworld_ubuntu:latest
[root@localhost docker_test]#
```

(6)镜像创建完成后，启动容器：

```
[root@localhost docker_test]# sudo docker run --name test helloworld_ubuntu:latest
Hello world!
[root@localhost docker_test]#
```

(7)出现”**Hello World！**”则实验运行成功。

五、实验数据（现象）处理分析

问题与解决

在本实验中，发现未能成功安装 **kvm**，查询网络发现有以下方法：

1. **检查KVM支持**: 首先，您应该确认您的硬件支持KVM，并且该功能已经在BIOS中启用。在Linux中，您可以通过执行以下命令来检查CPU是否支持硬件虚拟化：

```
egrep '(vmx|svm)' --color=always /proc/cpuinfo
```

如果有输出，并且标记了 `vmx` （Intel处理器）或 `svm` （AMD处理器），则您的CPU支持硬件虚拟化。

2. **检查KVM模块**: 确认KVM内核模块是否已经加载到系统中。可以使用以下命令来进行检查：

```
lsmod | grep kvm
```

如果看到 `kvm` 和 `kvm_intel` （对于Intel处理器）或 `kvm_amd` （对于AMD处理器）的相关模块输出，说明KVM模块已经加载。

如果没有加载，您可以通过以下命令尝试手动加载：

```
sudo modprobe kvm
sudo modprobe kvm_intel  # 对于Intel处理器
sudo modprobe kvm_amd    # 对于AMD处理器
```

3. **检查qemu-kvm安装**: 确保您已经安装了 `qemu-kvm` 包。可以用包管理器（如 `yum` 或 `apt` ）来安装它。

```
sudo yum install qemu-kvm  # 在基于RPM的系统上
sudo apt install qemu-kvm  # 在基于Debian的系统上
```

4. **修改虚拟机配置**: 如果硬件和KVM模块都没有问题，可能是XML配置文件中 `<type>` 标签设定有误。打开XML配置文件，找到表示虚拟化类型的 `<type>` 标签，并确保其值是 `kvm` 。例如：

```
<domain type='kvm'>
  ...
</domain>
```

5. **重启libvirtd服务**: 在有些情况下，重启 `libvirtd` 服务可以解决问题：

```
sudo systemctl restart libvirtd
```

6. **查看日志文件**: 如果在上述步骤后仍然有错误，您可以查看libvirt和qemu的日志文件，里面可能包含更具体的信息，帮助您找到问题所在。

尝试了先使用 **egrep -c '(vmx|svm)' /proc/cpuinfo** 检测是否支持虚拟化，结果返回 **0**，说明并不支持虚拟化

尝试直接安装 **kvm**

```
[root@localhost ~]# sudo dnf install qemu-kvm libvirt virt-manager bridge-utils
Last metadata expiration check: 0:00:10 ago on 2024年12月11日 星期三 13时38分44秒.
Package qemu-10:6.2.0-100.oe2203sp4.x86_64 is already installed.
Package libvirt-6.2.0-66.oe2203sp4.x86_64 is already installed.
Dependencies resolved.
================================================================================
 Package                 Architecture            Version
================================================================================
Installing:
 bridge-utils            x86_64                  1.7.1-2.oe2203sp4
 virt-manager            noarch                  2.1.0-10.oe2203sp4
Installing dependencies:
 at-spi2-atk-devel       x86_64                  2.38.0-4.oe2203sp4
 at-spi2-core-devel      x86_64                  2.42.0-4.oe2203sp4
 atk-devel               x86_64                  2.38.0-1.oe2203sp4
 autoconf                noarch                  2.71-7.oe2203sp4
 automake                noarch                  1.16.5-9.oe2203sp4
 brotli-devel            x86_64                  1.0.9-3.oe2203sp4
 bzip2-devel             x86_64                  1.0.8-6.oe2203sp4
```

安装成功后启动服务

```
[root@localhost ~]# sudo systemctl enable --now libvirtd
[root@localhost ~]#
```

```
[root@localhost ~]# sudo systemctl status libvirtd
● libvirtd.service - Virtualization daemon
     Loaded: loaded (/usr/lib/systemd/system/libvirtd.service; enabled; vendor preset: enabled)
     Active: active (running) since Wed 2024-12-11 13:39:57 CST; 30s ago
TriggeredBy: ● libvirtd-ro.socket
             ● libvirtd-admin.socket
             ● libvirtd.socket
       Docs: man:libvirtd(8)
             https://libvirt.org
   Main PID: 806478 (libvirtd)
      Tasks: 19 (limit: 32768)
     Memory: 45.7M
     CGroup: /system.slice/libvirtd.service
             ├─  1681 /usr/sbin/dnsmasq --conf-file=/var/lib/libvirt/dnsmasq/default.conf --leasefile-ro --dhcp-script=/usr/libexec/libvirt_leaseshelper
             ├─  1682 /usr/sbin/dnsmasq --conf-file=/var/lib/libvirt/dnsmasq/default.conf --leasefile-ro --dhcp-script=/usr/libexec/libvirt_leaseshelper
             └─806478 /usr/sbin/libvirtd --timeout 120

12月 11 13:39:57 localhost systemd[1]: Starting Virtualization daemon...
12月 11 13:39:57 localhost systemd[1]: Started Virtualization daemon.
12月 11 13:39:58 localhost dnsmasq[1681]: read /etc/hosts - 8 names
12月 11 13:39:58 localhost dnsmasq[1681]: read /var/lib/libvirt/dnsmasq/default.addnhosts - 0 names
12月 11 13:39:58 localhost dnsmasq-dhcp[1681]: read /var/lib/libvirt/dnsmasq/default.hostsfile
[root@localhost ~]#
```

查看虚拟机列表

```
[root@localhost ~]# virsh list --all
 Id   名称   状态
--------------------

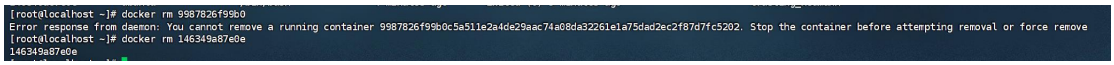
[root@localhost ~]#
```

将 **arrch64** 改为 **x86** 架构，仍然提示错误

```
[root@localhost ~]# virsh define vm_openEuler.xml
错误：从 vm_openEuler.xml 定义域失败
错误：不支持的配置：Emulator '/usr/libexec/qemu-kvm' does not support arch 'aarch64'

[root@localhost ~]# vi vm_openEuler.xml
[root@localhost ~]# virsh define vm_openEuler.xml
错误：从 vm_openEuler.xml 定义域失败
错误：不支持的配置：Emulator '/usr/libexec/qemu-kvm' does not support virt type 'kvm'
```

无法终止一个正在运行的容器，只有停止的容器才能被删除



实验讨论

(1) 第一个任务中，使用 **libvirt+xml** 配置文件的方法来搭建 **QEMU** 虚拟机运行环境，遇到的挑战和难点

在使用 **libvirt+xml** 配置文件配置 **QEMU** 虚拟机时，遇到的一些挑战和难点包括：

- **XML** 配置文件语法复杂：**libvirt** 配置文件是基于 **XML** 的，虽然语法不难，但在实际编写时，容易因为格式错误（如标签匹配、空格、换行等）导致虚拟机无法启动，调试起来较为麻烦。
- 虚拟机网络配置：配置虚拟机的网络时，需要正确设置网络桥接或虚拟网络，涉及的配置项比较多。网络模式的选择（如 **NAT** 或桥接）对于虚拟机的通信能力有很大影响，初次配置时容易出现网络连接问题。
- 硬件资源分配：在配置 **CPU**、内存、硬盘等硬件资源时，选择不当可能导致虚拟机性能不佳，甚至无法启动。资源分配需要根据宿主机的性能和虚拟机的需求来进行合理配置。
- 版本兼容问题：有时候不同版本的 **libvirt** 或 **QEMU** 可能存在兼容性问题，导致某些配置项在不同版本中无法正常工作，调试过程中需要确认版本兼容性。

解决方案：

- 使用现有的虚拟机模板文件，并参考官方文档或社区讨论来确认正确的配置。
- 在网络配置上，确保虚拟机与宿主机之间的连接配置正确，并仔细设置桥接网络或 NAT。
- 定期更新 libvirt 和 QEMU 的版本，以确保支持最新的特性和 bug 修复。

(2) 第二个任务中，在安装和使用 Docker 过程中遇到的问题及解决方案

在安装和使用 Docker 时，可能遇到的常见问题及解决方案如下：

- 权限问题：在安装 Docker 时，可能会遇到没有足够权限的问题，特别是在非 root 用户下执行命令时，常见错误为 "permission denied"。

解决方案：使用 sudo 提升权限，或将用户加入 Docker 组，通过 sudo usermod -aG docker <username> 来允许用户直接使用 Docker。

- 镜像下载失败：下载 Docker 镜像时，有时会遇到网络问题，导致镜像无法下载，尤其是在国内使用时，Docker Hub 的连接可能较慢。

解决方案：使用国内镜像源，例如阿里云、网易云等，配置 Docker 的镜像加速器。

- Docker 服务启动失败：安装 Docker 后，可能会遇到 Docker 服务无法启动的问题，常见错误是服务没有成功启动或启动时

发生冲突。

解决方案：检查 **Docker** 服务状态，使用 **systemctl status docker** 查看日志，并根据错误提示进行修复（如解决端口冲突、调整配置文件等）。

- 容器网络配置问题：在启动容器时，可能会遇到网络连接问题，比如容器无法访问外部网络。

解决方案：检查 **Docker** 网络配置，确保网络模式设置正确，或使用 **docker network inspect** 查看网络设置。

(3) 第三个任务中，使用 **Dockerfile** 创建基于 **Ubuntu** 镜像的 **Docker** 镜像时，遇到的问题及优化 **Dockerfile**

在编写 **Dockerfile** 时，遇到的挑战和问题包括：

- 层的过多或不必要的安装：**Docker** 镜像的每一层都会增加镜像的大小和构建时间。如果 **Dockerfile** 中安装了不必要的工具或文件，会导致镜像冗余。

优化方法：

  - 尽量合并多个命令，减少镜像层。例如，合并多个 **RUN** 命令为一个。
  - 使用 **&&** 在同一命令行中执行多个操作，从而减少镜像层数。

- 使用缓存问题：在 **Dockerfile** 中，每次构建都会使用缓存，这有时会导致更新的操作没有反映在镜像中，尤其是在构建中间层时。

解决方案：可以通过 **--no-cache** 选项来强制 **Docker** 不使用缓存，确保每次构建都是全新的。

- 镜像体积过大：如果 **Dockerfile** 没有进行合理的优化，生成的镜像体积可能会非常大，导致下载和部署变慢。

优化方法：

  - 使用轻量级的基础镜像（如 **alpine** 而非 **ubuntu**），来减少镜像体积。
  - 删除不必要的文件和缓存，例如，在安装完软件包后使用 **rm -rf /var/lib/apt/lists/*** 来清理缓存文件。

- 构建时间过长：有时由于构建步骤过多或者文件下载速度慢，构建时间可能很长。

优化方法：

  - 优化文件下载源，例如使用国内镜像源来加快包的安装。
  - 使用多阶段构建（**multi-stage builds**）来减少最终镜像的大小，并提高构建效率。

总结： 通过解决 **Dockerfile** 中的冗余层、优化基础镜像选择及文件清理等，可以有效提高镜像构建的效率和镜像的性能，减少构建时间和体积。

六、实验结论

通过本次实验，我深入理解了虚拟化和容器化技术。在搭建 **QEMU** 虚拟机时，成功解决了配置文件中的资源分配和网络问题；在 **Docker** 安装和使用过程中，解决了权限和网络配置等常见问题；通过编写和优化 **Dockerfile**，我成功构建了一个轻量级且高效的 **Ubuntu** 镜像。总体而言，实验增强了我在虚拟机管理、**Docker** 容器创建与优化方面的实践能力，提升了解决实际问题的技巧。

七、教师批阅意见：

成绩评定：

教师签字：

年　　月　　日

八、备注：